# Weekly Meeting – Testing der Benutzerregistrierung

Bereich: Testing & Validierung
Sprint: Sprint 1

## 1. Ziel des Testings

Ziel ist es sicherzustellen, dass das System fehlerhafte Eingaben bei der Benutzerregistrierung erkennt und passende Fehlermeldungen ausgibt.

## 2. Simple Test Cases

| TestCase No. | Category | Input | Output (Expected) |
|---|---|---|---|
| 1 | pos | Neue E-Mail + gültiges Passwort | Registrierung erfolgreich |
| 2 | neg | Bereits existierende E-Mail | Fehler: „E-Mail bereits vergeben“ |
| 3 | neg | Passwort = "123" | Fehler: „Passwort zu kurz“ |

## 3. Zugehörige Äquivalenzklassen

### Parameter: E-Mail-Adresse

| Äquivalenzklasse | Repräsentant |
|---|---|
| pos_ÄK: neue E-Mail-Adresse | test@mail.de |
| neg_ÄK: existierende E-Mail-Adresse | max@mail.de |
| neg_ÄK: ungültiges Format | testmail |

### Parameter: Passwort

| Äquivalenzklasse | Repräsentant |
|---|---|
| pos_ÄK: Passwort ≥ 8 Zeichen | Passwort123 |
| neg_ÄK: Passwort < 8 Zeichen | 123 |
| neg_ÄK: leeres Passwort | "" |

## 4. UML-Aktivitätsdiagramm (Beschreibung)

[Start]
↓
Benutzer gibt Registrierungsdaten ein
↓
Prüfung: E-Mail bereits vorhanden?
↓
Ja → Fehlermeldung „E-Mail bereits vergeben“
Nein ↓
Prüfung: Passwort lang genug?
↓
Nein → Fehlermeldung „Passwort zu kurz“

Ja ↓
Benutzer registrieren
↓
Erfolgsmeldung
↓
[Ende]

## 5- JUnit-Testfälle zur Benutzerregistrierung

```
assertTrue(login("test@mail.de", "Passwort123"));

assertFalse(login("test@mail.de", "123"));

assertFalse(login("max@mail.de", "Passwort123"));

assertFalse(login("test@mail.de", ""));
```